WEATHER APP CASE STUDY

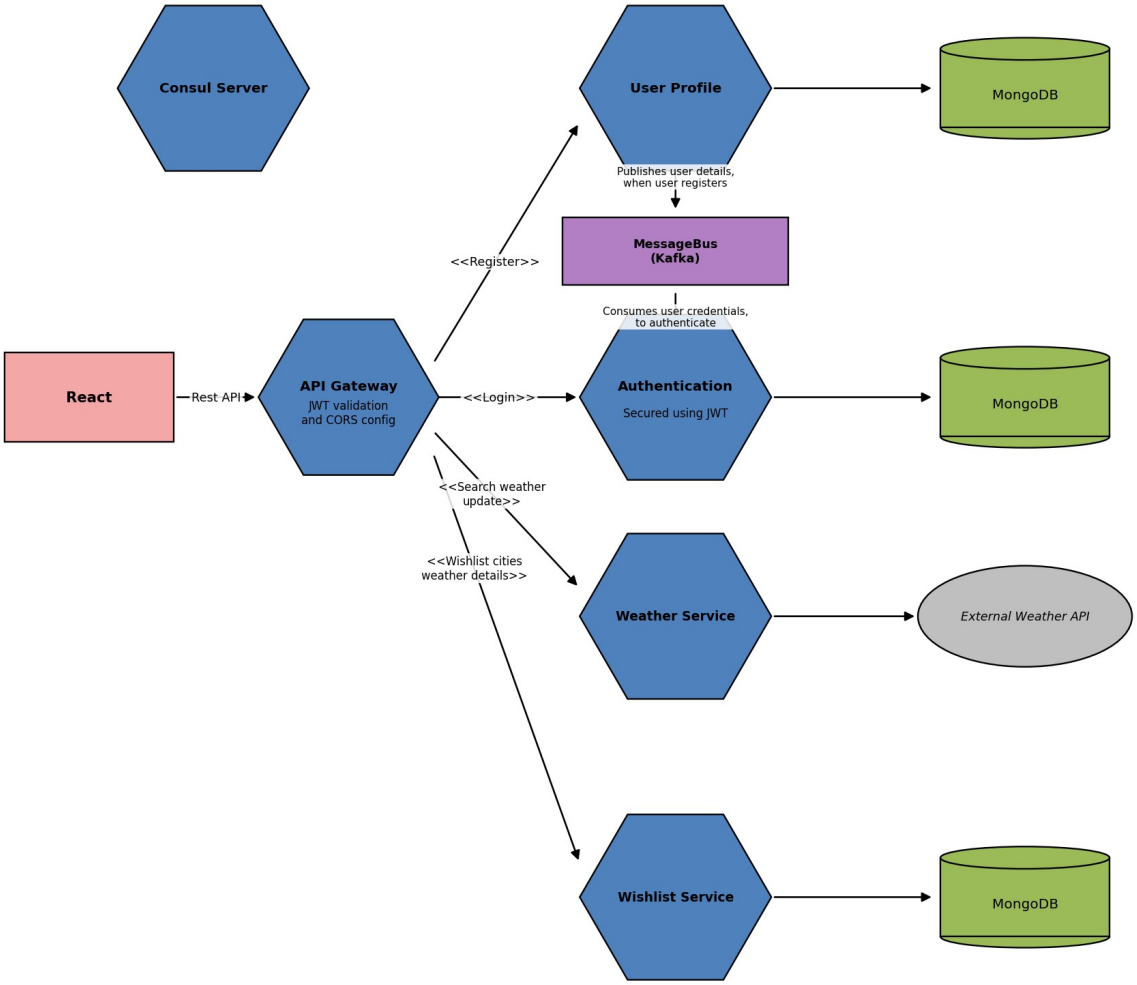
Name of the Project	Weather App
Objective	Develop a Weather Application that allows users to view weather forecast details for a particular city, and save the cities to favorites/wishlist. The application needs to fetch weather details by registering with the following API and get the API Key required to call the API. https://openweathermap.org/api Sample API: api.openweathermap.org/data/2.5/weather?id={city id}&appid={API key}
Functional Requirements	User Interface (UI) should achieve the following: ● User Registration ● User Login ● View weather forecast details of selected city ● Search for a city ● Add selected city to your favorite list ● View favorite cities ● UI should be responsive which can run smoothly on various devices ● The UI should be appealing and user friendly
Non-functional Requirements	● The app should be able to load weather details of cities quickly and smoothly, even on low-end devices. ● The app should be able to handle a large number of users without slowing down or crashing. ● The app should be easy to use and navigate, even for users with no prior experience with weather apps. ● The app should protect user data from unauthorized access, modification, or deletion.
Technical Requirements	● Application should be developed using Microservices in the Backend. JWT tokens to be used for securing the Backend. ● Frontend should be developed using React. ● Microservice patterns like API Gateway, Service Discovery, Microservice communication. ● Comprehensive Unit tests and integration tests with coverage should be implemented to validate the functionality of the Application. ● Application should be integrated with databases. ● Implement Documentation of API using Swagger/Open API.
Tools and Technologies to be used	SCM : Gitlab Backend : NodeJS/ExpressJS Frontend : React Data Store : MongoDB Testing : Chai, Mocha, Jest, React Testing Library/Enzyme CodeQuality : Sonar Lint API Documentation : Swagger Message Bus : Kafka

User Stories

1	**As a user, I should be able to register with the application so that I can login and use the functionalities of the application.**

2	**As a user, I should be able to login with my username and password in order to access the functionalities of the application.**
3	**As a user, I should be able to search a city to check its weather details using Third Party API.**
4	**As a user, I should be able to save cities to a wishlist/favourite so that I can access them later.**
5	**As a user, I should be able to access cities saved to my wishlist/favourite.**
6	**As a user, I should be able to delete cities saved to my wishlist/favourite.**

High Level Architecture Diagram



The responsibilities of the microservices in the above figure are as follows:

- **User Profile Service:** This Service is responsible for storing user registration details. The Service publishes the user credentials sent as part of registration to the message bus and stores the remaining user profile information in the database.
- **Authentication Service:** This Service is responsible for consuming user credential from the message bus and storing it in the database. When a user logs in, this service validates the login credentials against the credentials stored in the database. If the credentials match, this service generates a JWT token and sends it back as a response, else an error message is sent.

- **Weather Service:** This Service is responsible for accessing an external Weather API to fetch weather details of a city on the search criteria coming in as a request and returning back the matching weather details as response.
- **Wishlist Service:** This Service is responsible for storing cities bookmarked by users in the database.
- **API Gateway:** This Service acts as the entry point of the system. It intercepts all the requests and validates the JWT Token before routing it to the appropriate microservices.
- **Consul Server:** This Service acts as a service registry where all the other microservices register during startup for discoverability.

Recommended Steps to complete the Case Study

Step 1: Understand the Case Study

Step 2: Identify the Data Model and draw the data flow diagram

Step 3: Draw the UI Wireframes

Step 4: Create the Boilerplate

Step 5: Implement and write test cases for the backend

Step 6: Implement and write test cases for the frontend

Step 7: Integrate the frontend with the backend